

Experiment 3

AIM: Design Employee Payroll System using Inheritance, Interface & Polymorphism.

Source: Practical 3.cs

Code:

```
using System;

namespace Practical2
{
    class Expense
    {
        public int expenseId;
        public string expenseName;
        public DateTime expenseDate;
        public double amount;

        public Expense()
        {
            Console.WriteLine("Expense Object Created");
        }

        public void GetDetails()
        {
            Console.Write("Enter Expense ID: ");
            expenseId = int.Parse(Console.ReadLine());

            if (expenseId <= 0)
                throw new ArgumentException("Expense ID must be greater than 0.");

            Console.Write("Enter Expense Name: ");
            expenseName = Console.ReadLine();

            if (string.IsNullOrEmpty(expenseName))
                throw new ArgumentNullException("Expense Name", "Expense Name cannot be empty.");

            Console.Write("Enter Expense Date (dd/MM/yyyy): ");
            expenseDate = DateTime.ParseExact(Console.ReadLine(), "dd/MM/yyyy", null);

            if (expenseDate > DateTime.Now)
                throw new ArgumentOutOfRangeException("Expense Date", "Expense Date cannot be in the future.");
        }
    }
}
```

```
        Console.WriteLine("Enter Expense Amount: ");
        amount = double.Parse(Console.ReadLine());

        if (amount <= 0)
            throw new ArgumentOutOfRangeException("Amount", "Expense Amount must be
greater than zero.");

        if (amount > 10000000)
            throw new OverflowException("Expense Amount is too large.");
    }

    public void DisplayExpense()
    {
        Console.WriteLine("\nExpense Details");
        Console.WriteLine("Expense ID      : " + expenseId);
        Console.WriteLine("Expense Name   : " + expenseName);
        Console.WriteLine("Expense Date   : " + expenseDate.ToString("dd/MM/yyyy"));
        Console.WriteLine("Expense Amount : " + amount);
    }
}

class Program
{
    static void Main(string[] args)
    {
        Expense exp = new Expense();

        try
        {
            exp.GetDetails();
            exp.DisplayExpense();
        }

        catch (FormatException)
        {
            Console.WriteLine("Invalid format! Please enter numbers where required
and date in dd/MM/yyyy format.");
        }

        catch (ArgumentNullException ex)
        {
            Console.WriteLine(ex.Message);
        }

        catch (ArgumentException ex)
```

```
{
    Console.WriteLine(ex.Message);
}

catch (ArgumentOutOfRangeException ex)
{
    Console.WriteLine(ex.Message);
}

catch (OverflowException ex)
{
    Console.WriteLine(ex.Message);
}

finally
{
    Console.WriteLine("\nExpense Tracking Completed.");
}

Console.ReadKey();
}
}
```

Output:

```
Expense Object Created
Enter Expense ID: -1
Expense ID must be greater than 0.

Expense Tracking Completed.
```

```
Expense Object Created
Enter Expense ID: 57
Enter Expense Name: 1
Enter Expense Date (dd/MM/yyyy): 26/12/2026
Expense Date cannot be in the future.
Parameter name: Expense Date

Expense Tracking Completed.
```

```
Expense Object Created
Enter Expense ID: 2
Enter Expense Name: 2
Enter Expense Date (dd/MM/yyyy): 26/06/26
Invalid format! Please enter numbers where required and date in dd/MM/yyyy format.

Expense Tracking Completed.

C:\Users\student\Desktop\.Net\practical-4\bin\Debug\practical-4.exe (process 18224)
To automatically close the console when debugging stops, enable Tools->Options->Debu
Press any key to close this window . . .
```

GitHub URL: <https://github.com/jayvisana/.NET-Practicals-.git>